

Prompt Engineering

Context Management for Reliable LLM Outputs

Dr. Papa-Séga WADE

AIMS Senegal | Module 2 - Session 5

MODULE 2 - Session 5: prompting, context, and reliability

"Prompt like an engineer: specify, constrain, verify, iterate"

Where We Stopped in Session 4

Recap of Module 1

We now understand what LLMs are, how they are trained (pretraining, SFT, alignment), and how to choose one (open vs closed, cost, quantization, reasoning, speech). The model is selected. The infrastructure is sized. The question now is: **how do we actually use it well?**

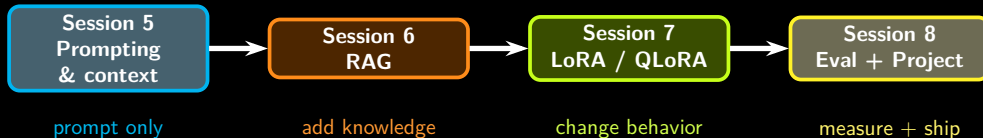


We are here

Module 2: Using and Adapting an LLM

Goal of the module

Move from understanding to practice. Use an LLM effectively (prompting), connect it to external knowledge (RAG), and adapt it to a domain (fine-tuning).



Decision tree (to keep in mind for the whole module)

If prompting is enough → use prompting. If you need specific knowledge → RAG. If you need to change style or behavior → fine-tuning.

Baseline Objective

By the end of this session

You should master prompting techniques for reliable LLM outputs and know how to manage long-context inputs.

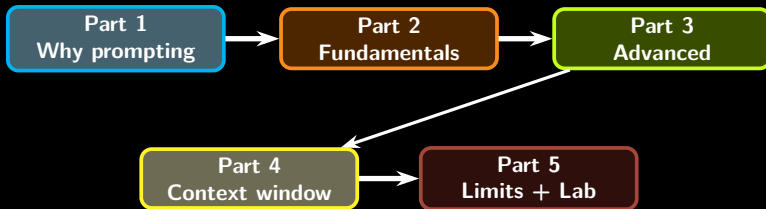
You will be able to

- write zero-shot, few-shot, and role prompts,
- enforce output formats (JSON, table...),
- apply chain-of-thought and self-consistency,
- structure long prompts strategically,
- recognize when prompting is not enough.

Guiding question

For each AIMS task, what is the simplest prompt that gives a reliable, reproducible answer?

Session Roadmap



Core message

Same model, different prompt, radically different result. A good prompt is the cheapest engineering lever you have.

Same Model, Different Prompt, Different Result

Vague prompt

"Tell me about attention."

Likely output

Generic, surface-level paragraph. Could be about psychology, neural networks, or marketing. Not reproducible.

Engineered prompt

"You are an NLP instructor. In 5 bullets, explain self-attention to a Master 2 student who knows linear algebra. Use one analogy."

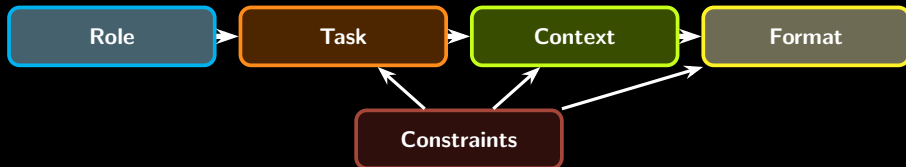
Likely output

Targeted, structured, length-controlled, calibrated to the audience. Reproducible across runs.

Key idea

The prompt is not a question. It is a **specification of the desired output**.

Anatomy of a Good Prompt



What each block carries

- **Role**: who the model should be
- **Task**: what to produce
- **Context**: facts the model needs
- **Format**: shape of the output

Constraints typically include

- length (e.g. 150 words),
- language (English / French / Wolof),
- what **not** to do (no speculation, no code).

Zero-Shot Prompting

Definition

Ask the model to perform a task with **no examples**. The instruction alone must be enough.

When it works

- the task is common in pretraining (translation, summary, classification),
- the instruction is unambiguous,
- the output format is simple.

```
1 Translate the following sentence
2 from English to French:
3
4 "The course starts on May 18."
```

Limitation

Performance is unstable on rare formats, domain-specific tasks, or when the model must follow a precise structure.

Few-Shot Prompting

Definition

Provide **a few input/output examples** before the real query. The model imitates the pattern.

Why it helps

- disambiguates the task,
- teaches the exact output format,
- compensates for missing instruction-following on smaller models.

Practical rule

3 to 5 examples is usually enough. Too many examples waste context.

```
1 Extract the country of birth.  
2  
3 Text: "Marie was born in Dakar."  
4 Country: Senegal  
5  
6 Text: "Ahmed grew up in Cairo."  
7 Country: Egypt  
8  
9 Text: "Lina was born in Bamako."  
10 Country:
```

Role Prompting

Definition

Give the model a persona that biases its style, vocabulary, and depth.

```
1 You are a senior NLP researcher
2 reviewing a Master thesis.
3 Be precise, point out 3 specific
4 weaknesses, and suggest concrete
5 fixes.
6 Avoid generic praise.
```

Useful roles

- **teacher**: explains step by step
- **reviewer**: critical, structured
- **translator**: faithful, terminologically correct
- **junior dev**: writes simple code with comments

Caution

Roles bias **tone and depth**, not factual accuracy. They do not fix hallucinations.

Format Constraints: JSON, Table, Bullets

Why structured output matters

Structured output can be parsed by code. Unstructured prose often requires regex, retries, or another LLM call.

JSON example

```
1 Return a JSON object with keys:
2 {
3   "summary": str (max 3 sentences),
4   "topics": list[str],
5   "language": "en" | "fr"
6 }
7 No prose outside the JSON.
```

Table example

```
1 Output a markdown table with
2 columns: term | definition | example.
3 One row per concept.
4 No introduction.
```

Engineering tip

End constrained prompts with **negative instructions**: "no preamble", "no explanation", "no markdown fences".

Chain-of-Thought (CoT)

Idea

Ask the model to **reason step by step** before the final answer. Reasoning tokens often help on math, logic, and multi-hop tasks.

```
1 Q: A class has 23 students.  
2 Each pays 4500 FCFA. The total  
3 is split between 3 teachers.  
4 How much per teacher?  
5  
6 Let's reason step by step,  
7 then write the final number on  
8 a line starting with "ANSWER:".
```

Two flavors

- **Zero-shot CoT**: append "Let's think step by step."
- **Few-shot CoT**: provide examples whose answers include the reasoning.

When NOT to use CoT

Avoid it for trivial questions, format-only tasks, and latency-critical applications. CoT increases token cost.

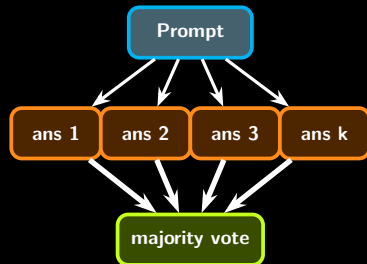
Self-Consistency

Idea

Sample **k independent CoT answers** (temperature > 0), then take the majority vote. This reduces variance from one risky reasoning chain.

When it pays off

- math word problems,
- logical reasoning,
- multi-step extraction.



Cost reminder

k samples means $k \times$ cost. Pick $k \in \{3, 5, 7\}$; beyond that, gains plateau.

Quick Comparison of Techniques

Technique	Best for	Token cost	Reliability
Zero-shot	common tasks	low	medium
Few-shot	format / rare tasks	medium	good
Role	tone, depth, audience	low	medium
CoT	math, logic, multi-hop	high	good
Self-consistency	high-stakes reasoning	very high	best

Heuristic

Start with the cheapest reliable technique. Escalate only when the task fails reproducibly.

The Context Window: From 8K to 1M+ Tokens

What it is

The maximum number of tokens (input + output) the model can process in one call.

2025-2026 reality

- small models: 8K to 32K
- modern open models: 128K
- frontier models: 1M+ (Gemini, Claude long-context)

Three things that consume tokens

- the system prompt and instructions,
- the user content (documents, examples),
- the model's own output.

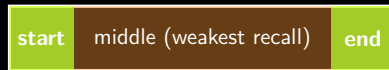
Cost note

Long context $\neq$ free. You pay per token, and latency grows with input length.

The "Lost in the Middle" Effect

Empirical finding (Liu et al., 2023)

When relevant information is placed in the **middle** of a long context, models retrieve it less reliably than when it is at the **beginning** or **end**.



place key info
at the edges

Practical rule

- put critical instructions **first**,
- put critical evidence **last**,
- never assume the middle is safe.

Structuring a Long Prompt

A reliable layout for long inputs

- 1 **System role and task** (top)
- 2 **Output format and constraints**
- 3 **Optional few-shot examples**
- 4 **Reference documents** (clearly delimited)
- 5 **Final instruction restating the task**

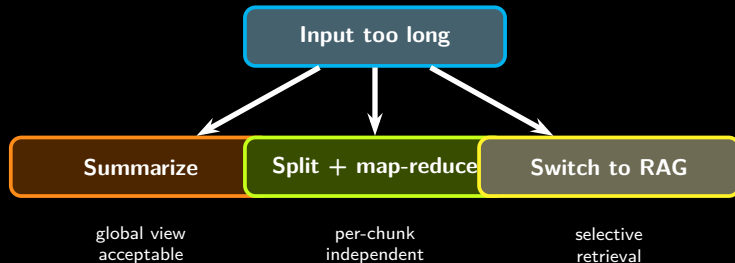
Why repeat the task at the end

Restating the instruction **after** the documents anchors the model on what to do, mitigating the lost-in-the-middle effect.

Delimiters that work

DOCUMENT 1 ###, <doc>...</doc>, triple backticks. Be consistent.

When to Summarize, When to Split



Decision rule

Summarize if you can lose detail. Split if chunks are independent. Switch to RAG (Session 6) if the corpus is large or grows over time.

Limits of Prompting

Hallucinations

The model generates plausible but **false** content with full confidence.

Knowledge cutoff

The model does not know events, papers, or APIs released **after** its training data.

Context window

Even 1M tokens is finite. Long corpora and growing knowledge bases do not fit.

What this implies

Prompting alone cannot give the model new factual knowledge or persistent behavior changes.

This is exactly what the rest of Module 2 addresses.



Lab: 4 AIMS Tasks

Goal

For each task, write three prompts (zero-shot, few-shot, CoT) and compare outputs on the same model.

Tasks

- 1 Summarize a scientific abstract (3 sentences).
- 2 Explain a math concept (e.g. softmax) to a Master 1 student.
- 3 Generate Python code: read a CSV and plot a histogram with matplotlib.
- 4 Extract structured info from a French text into JSON (name, role, organization).

Comparison grid

- correctness,
- format compliance,
- reproducibility (run twice),
- token cost.

Deliverable

One notebook per pair, pushed to GitHub Classroom.

Lab Skeleton (Python + Hugging Face)

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 def run(prompt, **kw):
6     return pipe(prompt, max_new_tokens=300, do_sample=False, **kw)[0]["generated_text"]
7
8 # Task 1 - zero-shot
9 zero = "Summarize the following abstract in 3 sentences:\n\n" + abstract
10
11 # Task 1 - few-shot
12 few = ("Abstract: ...\nSummary: ...\n\n"
13        "Abstract: ...\nSummary: ...\n\n"
14        f"Abstract: {abstract}\nSummary:")
15
16 # Task 1 - chain-of-thought
17 cot = ("Summarize the abstract below.\n"
18        "First list the 3 key claims, then write the 3-sentence summary.\n\n"
19        f"Abstract: {abstract}")
20
21 for label, p in [("zero", zero), ("few", few), ("cot", cot)]:
22     print(label, "->", run(p)[:400])
```

Advanced Extension: Self-Consistency Pipeline

Goal

Implement majority voting over $k = 5$ CoT samples on a small math benchmark (e.g. 10 word problems). Measure accuracy vs the single-sample baseline.

Steps

- 1 run CoT prompt with `temperature=0.7, k=5`
- 2 parse the final number from each response
- 3 return `Counter(answers).most_common(1)`
- 4 compare with deterministic single sample

Expected outcome

Self-consistency typically lifts accuracy by 5 to 15 points on arithmetic word problems, at $5\times$ token cost.

What to Remember

- 1 A prompt is a **specification of the desired output**, not a question.
- 2 Anatomy: role, task, context, format, constraints.
- 3 Escalate techniques only as needed: zero-shot → few-shot → CoT → self-consistency.
- 4 Place critical information at the **edges** of long prompts.
- 5 Prompting cannot fix missing knowledge or stale facts. RAG and fine-tuning will.

Key Resources

Core references

- Prompt Engineering Guide (promptingguide.ai)
- Wei et al., *Chain-of-Thought Prompting* (2022)
- Wang et al., *Self-Consistency* (2022)
- Liu et al., *Lost in the Middle* (2023)

Practical

- Hugging Face transformers pipelines
- OpenAI / Anthropic prompt cookbooks
- LangChain prompt templates

Bridge to Session 6: RAG

Next step

Prompting reaches its ceiling when the model needs information it does not know. In Session 6, we connect the LLM to external documents through Retrieval-Augmented Generation.



Questions & Discussion

Prompt like an engineer: specify, constrain, verify, iterate.

When prompting reaches its ceiling, move to retrieval for missing knowledge and fine-tuning for persistent behavior.

Dr. Papa-Séga WADE

AIMS Senegal | GAAI-AIMS